```
# Initialize Otter
import otter
grader = otter.Notebook("hw6.ipynb")
```

```
---------------------------------------------------------------------------
ValueError Traceback (most recent call last)
/tmp/ipykernel_83432/2111977886.py in <cell line: 0>()
 1 # Initialize Otter
 2 import otter
----> 3 grader = otter.Notebook("hw6.ipynb")

                                    2 frames
/usr/local/lib/python3.12/dist-packages/otter/check/notebook.py in __init__(self, nb_path, tests_dir,
tests_url_prefix)
 105
 106 if self.interpreter is IPythonInterpreter.COLAB and not os.path.isdir(tests_dir): --> 107
raise ValueError(f"Tests directory {tests_dir} does not exist")
 108
 109 cls = type(self)

ValueError: Tests directory ./tests does not exist
```

keyboard_arrow_down CPSC 330 - Applied Machine Learning

Homework 6: Clustering

Associated lectures: [Lectures 14 and 15]

Due date: See [deliverable due dates].

keyboard_arrow_down Imports

```
import os
from hashlib import sha1

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

%matplotlib inline
pd.set_option("display.max_colwidth", 0)
```

keyboard_arrow_down Instructions

rubric={points}

You will earn points for following these instructions and successfully submitting your work on Gradescope.

Group wotk instructions

You may work with a partner on this homework and submit your assignment as a group. Below are some instructions on working as a group.

The maximum group size is 2.

Use group work as an opportunity to collaborate and learn new things from each other.

Be respectful to each other and make sure you understand all the concepts in the assignment well.

It's your responsibility to make sure that the assignment is submitted by one of the group members before the deadline.

You can find the instructions on how to do group submission on Gradescope here.

If you would like to use late tokens for the homework, all group members must have the necessary late tokens available. Please note that the late tokens will be counted for all members of the group.

General submission instructions

Please **read carefully** **Use of Generative AI policy** before starting the homework assignment.

Run all cells before submitting: Go to `Kernel -> Restart Kernel and Clear All Outputs` , then select `Run -> Run All Cells` . This ensures your notebook runs cleanly from start to finish without errors.

Submit your files on Gradescope.

Upload only your `.ipynb` file **with outputs displayed** and any required output files.

Do **not** submit other files from your repository.

If you need help, see the Gradescope Student Guide.

Check that outputs render properly.

Make sure all plots and outputs appear in your submission.

If your `.ipynb` file is too large and doesn't render on Gradescope, also upload a PDF or HTML version so the TAs can view your work.

Keep execution order clean.

Execution numbers must start at "1" and increase in order.

Notebooks without visible outputs may not be graded.

Out-of-order or missing execution numbers may result in mark deductions.

Follow course submission guidelines: Review the [CPSC 330 homework instructions] for detailed guidance on completing and submitting assignments.

Points: 4

keyboard_arrow_down Exercise 1: Document clustering warm-up

In this homework, we will explore a popular application of clustering called **document clustering**. A large amount of unlabeled text data is available out there (e.g., news, recipes, online Q&A, tweets), and clustering is a commonly used technique to organize this data in a meaningful way.

As a warm up, in this exercise you will cluster sentences from a toy corpus. Later in the homework you will work with a real corpus.

The code below extracts introductory sentences of Wikipedia articles on a set of queries. To run the code successfully, you will need the `wikipedia` package installed in the course environment.

```
conda activate cpsc330
conda install -c conda-forge wikipedia
```

Your tasks:

Run the code below which

extracts content of Wikipedia articles on a set of queries

tokenizes the text (i.e., separates sentences) and
stores the 2nd sentence in each article as a document representing that article

Feel free to experiment with Wikipedia queries of your choice. But stick to the provided list for the final submission so that it's easier for the TAs to grade your submission.

For tokenization we are using the `nltk` package. If you do not have this package in the course environment, you will have to install it.

```
conda activate cpsc330
conda install -c anaconda nltk
```

Even if you have the package installed via the course `conda` environment, you might have to download `nltk` pre-trained models, which can be done with the code below.

```
import nltk

nltk.download("punkt")
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
True
```

```
!pip install wikipedia
```

```
Requirement already satisfied: wikipedia in /usr/local/lib/python3.12/dist-packages (1.4.0)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from wikipedia)
(4.13 Requirement already satisfied: requests<3.0.0,>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from
wikipedi Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
(from reques Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from
requests<3.0.0,>=2 Requirement already satisfied: urllib3<3,>=1.21.1 in
```

```
/usr/local/lib/python3.12/dist-packages (from requests<3.0 Requirement already satisfied: certifi>=2017.4.17
in /usr/local/lib/python3.12/dist-packages (from requests<3.0 Requirement already satisfied: soupsieve>1.2 in
/usr/local/lib/python3.12/dist-packages (from beautifulsoup4->w Requirement already satisfied:
typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beauti
```

```
import wikipedia
import string
from nltk.tokenize import sent_tokenize, word_tokenize
```



```
queries = [
    "Artificial Intelligence",
    "Deep learning",
    "Unsupervised learning",
    "Quantum Computer",
    "Environmental protection",
    "Climate Change",
    "Renewable Energy",
    "Biodiversity",
    "French Cuisine",
    "Bread food",
    "Dumpling food",
]
wiki_dict = {"wiki query": [], "text": [], "n_words": []}
remove_tokens = list(string.punctuation) + ['``', '’', '`', 'br', '"', "”", "'", "'s", "(", ")", "[", "]"]

# Running this code might take some time.
for query in queries:
    try:
        # Attempt to fetch the page content
        page_content = wikipedia.page(query).content
    except wikipedia.exceptions.PageError:
        print(f"Page not found for query: {query}. Skipping...")
        continue
    except wikipedia.exceptions.DisambiguationError as e:
        print(f"Query: {query} led to a disambiguation page. Choosing the first option: {e.options[0]}")
        page_content = wikipedia.page(e.options[0]).content

    text = sent_tokenize(page_content)[0]
```

```
    tokenized = word_tokenize(text)
    text_pp = [token.lower() for token in tokenized if token.lower() not in remove_tokens]
    wiki_dict["n_words"].append(len(text_pp))
    wiki_dict["text"].append(" ".join(text_pp))
    wiki_dict["wiki query"].append(query)

wiki_df = pd.DataFrame(wiki_dict)
wiki_df
```


	wiki query	text	n_words
0	Artificial Intelligence	artificial intelligence ai is the capability of computational systems to perform tasks typically associated with human intelligence such as learning reasoning problem-solving perception and decision-making	25
1	Deep learning	in machine learning deep learning dl focuses on utilizing multilayered neural networks to perform tasks such as classification regression and representation learning [22]	
2	Unsupervised learning	in machine learning supervised learning sl is a type of machine learning paradigm where an algorithm learns to map input data to a specific output based on example input-output pairs [30]	
3	Quantum Computer	a quantum computer is a real or theoretical computer that exploits superposed and entangled states [15]	
4	Environmental protection	environmental protection or environment protection refers to the taking of measures to protecting the natural environment prevent pollution and maintain ecological balance [22]	
5	Climate Change	present-day climate change includes both global warming—the ongoing increase in global average temperature—and its wider effects on earth climate system [20]	
6	Renewable Energy	renewable energy also called green energy is energy made	

from renewable natural resources that are replenished on a human timescale 20

7 Biodiversity biologists most often define biodiversity as the totality of genes species and ecosystems of a region 16

8 French Cuisine french cuisine is the cooking traditions and practices of france 10 **9** Bread food bread is a baked food product made from water flour and often yeast 13 **10** Dumpling food manti is a type of dumpling mainly found in turkish cuisine armenian cuisine and central asian cuisine but also in west asia south caucasus and the balkans 27

Next steps: New interactive sheet

Our toy corpus has six toy documents (`text` column in the dataframe) extracted from Wikipedia queries.

keyboard_arrow_down 1.1 How many clusters?

rubric={reasoning}

Your tasks:

If tasked with manually clustering documents from this toy corpus, how many clusters would you identify, and what labels would you assign to each cluster?

Solution_1.1

Points: 1

Based on the wikipedia titles I would identify three salient categories which are machine learning (artificial intelligence, deep learning, unsupervised learning, quantum computer), environmental studies (environmental protection, climate change, renewable energy, biodiversity) and cuisines/food (french cuisine, bread food, dumpling food).

keyboard_arrow_down 1.2 K-Means with bag-of-words representation

rubric={accuracy}

In the lecture, we explored how **data representation significantly impacts clustering quality**. We saw that replacing a **flattened image representation** with feature vectors extracted from **pre-trained models** greatly improved clustering results.

What about text data? In previous machine learning courses, we have used the **bag-of-words (BoW) representation** to numerically encode text, where each document is represented with a vector of word frequencies.

In this exercise, you will **cluster documents** using this **simplistic text representation**.

Your tasks:

1. Use `CountVectorizer` with its **default settings** to transform the `text` column in `wiki_df` into a numerical representation.
2. Use `KMeans` to cluster the encoded documents and store the cluster labels in `kmeans_bow_labels` .

   Set `random_state=42` for **reproducibility**.

   Use the **number of clusters** (`n_clusters`) identified in the **previous exercise**.

Solution_1.2

Points: 2

```
from sklearn.feature_extraction.text import CountVectorizer

vec = CountVectorizer()
X_text = vec.fit_transform(wiki_df["text"])
```

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters = 3, random_state = 42)
kmeans.fit(X_text)
```



```
kmeans_bow_labels = kmeans.labels_
```

```
wiki_df["bow_kmeans"] = kmeans_bow_labels
wiki_df
```

	wiki	query	text	n_words	bow_kmeans
0	Artificial Intelligence				
1	Deep learning				
2	Unsupervised learning				

example input-output pairs

3 Quantum artificial intelligence ai is the capability of computational systems to perform tasks typically associated with human intelligence such as learning reasoning problem-solving perception and decision-making

in machine learning deep learning dl focuses on utilizing multilayered neural networks to perform tasks such as classification regression and representation learning

a quantum computer is a real or theoretical computer that exploits

25 1 22 2 30 2

in machine learning supervised learning sl is a type of machine learning paradigm where an algorithm learns to map input data to a specific output based on

Computer

superposed and entangled states 15 1

environmental protection or environment protection refers to the taking of

4 Environmental protection

environment prevent pollution and maintain ecological balance

22 0 20 1

5 Climate Change

present-day climate change includes both global warming—the ongoing increase in global average temperature—and its wider effects on earth climate system

6 Renewable

renewable energy also called green energy is energy made from renewable

measures to protecting the natural

Energy

timescale 20 1

natural resources that are replenished on a human

7 Biodiversity biologists most often define biodiversity as the totality of genes species and ecosystems of a region 16 1

8 French Cuisine french cuisine is the cooking traditions and practices of france 10 1 **9** Bread food bread is a baked food

product made from water flour and often yeast 13 1 manti is a type of dumpling mainly found in turkish cuisine armenian cuisine

10 Dumpling food and central asian cuisine but also in west asia south caucasus and the balkans 27 1

Next steps: New interactive sheet

keyboard_arrow_down 1.3 K-Means with sentence embedding representation

rubric={accuracy}

In the previous exercise, you may have noticed that clustering results were suboptimal. While the bag-of-words (BoW) representation is useful, it has significant limitations. It does not capture word order or context effectively.

To improve text representation, we can use pre-trained models, similar to those used for image data, to extract richer, context-aware embeddings through transfer learning. In this lab, we will use the **pre-trained model** `'all-MiniLM-L6-v2'`, available via the Sentence Transformers package.

This deep learning model generates **dense, fixed-length vector representations** of sentences, capturing **semantic meaning and contextual information**. These representations are particularly effective for **clustering and semantic similarity tasks**. If you're interested in exploring more **pre-trained models**, refer to the full list here. We will explore these representations further in **DSCI 575**.

Your tasks:

1. Run the code below to create sentence embedding representation of documents in our toy corpus.
2. Apply `KMeans` to the **sentence embeddings** (`emb_sents`) and store the cluster labels in `kmeans_emb_labels` . Use the following parameters:

   `random_state=42` (for reproducibility)

   `n_clusters` =the number of clusters you identified in 1.1

Note

The code below might throw a warning. You may ignore it for the purpose of this lab.

```
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer("all-MiniLM-L6-v2")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or
datasets.  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate li
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED | |

Notes:
   - UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical a
```

```
emb_sents = embedder.encode(wiki_df["text"].tolist())
emb_sent_df = pd.DataFrame(emb_sents, index=wiki_df.index)
emb_sent_df
```

0 1 2 3 4 5 6 7 8 9 ...

0 -0.005857 -0.004795 -0.000976 0.011121 0.005294 -0.061630 0.062072 0.033673 -0.035251 0.074072 ... 0 **1** -0.122905 -0.092450 0.085379 -0.009761 0.001422 0.069146 -0.046654 -0.069960 -0.021858 -0.052803 ... 0 **2** -0.056721 -0.049697 -0.014780 0.022572 0.051773 0.007456 0.008632 -0.074847 -0.065573 -0.027284 ... 0 **3** -0.071932 -0.006760 -0.083421 0.022650 -0.116415 0.034988 -0.055792 -0.045055 -0.017993 0.020915 ... -0 **4** -0.012700 0.101830 0.066676 -0.007987 0.140040 0.041627 0.035495 -0.050723 -0.037633 0.055886 ... -0 **5** -0.014444 0.022869 0.145982 0.054865 0.085204 -0.014689 -0.102757 0.016853 -0.023799 0.021148 ... 0 **6** -0.044869 0.076882 -0.012749 0.033335 0.067894 -0.002326 0.031620 -0.021576 0.046619 0.005089 ... -0 **7** 0.027811 0.042017 -0.034509 -0.029904 0.008833 0.021378 -0.063310 0.014000 0.069925 0.000224 ... -0 **8** 0.026616 -0.019929 -0.008049 -0.025907 -0.055372 0.011235 -0.024538 -0.065724 0.058374 -0.062895 ... -0 **9** -0.014786 0.055887 -0.020880 0.005313 -0.025037 0.040479 0.035260 0.024938 0.017546 0.072433 ... -0 **10** -0.017367 0.046664 -0.089176 0.010790 -0.084132 -0.039986 0.033869 -0.100715 -0.010708 -0.068042 ... 0 11 rows × 384 columns

Solution_1.3

Points: 2

```
kmeans = KMeans(n_clusters = 3, random_state = 42)
kmeans.fit(emb_sent_df)
```

▾ KMeans i ?

```
KMeans(n_clusters=3, random_state=42)
```

```
kmeans_emb_labels = kmeans.labels_
```



```
wiki_df["emb_kmeans"] = kmeans_emb_labels
wiki_df
```

Next steps: New interactive sheet

wiki query text n_words bow_kmeans emb_kmeans artificial intelligence ai is the capability of computational

0 Artificial Intelligence

associated with human intelligence such as learning reasoning problem-solving perception and decision-making

where an algorithm learns to map input data

25 1 2

1 Deep learning **2** Unsupervised

in machine learning deep learning dl focuses on utilizing multilayered neural networks to perform tasks such as classification regression and representation learning

22 2 2 30 2 2

systems to perform tasks typically

in machine learning supervised learning sl is a type of machine learning paradigm

keyboard_arrow_down 1.4 DBSCAN with sentence embedding representation and cosine distance

learning

to a specific output based on example input-output pairs

a quantum computer is a real or theoretical computer that

exploits superposed and entangled states 15 1 2

3 Quantum

rubric={accuracy}

Computer

Now, let's apply `DBSCAN` to our toy dataset.

4 Environmental

environmental protection or environment

protection refers to the taking of measures to protecting the natural environment

22 0 0

Unlike K-Means, which relies on Euclidean distance due to its use of centroids, DBSCAN allows us to experiment with

protection

prevent pollution and maintain

ecological balance

different distance metrics. For text data, **cosine similarity** (or its complement, **cosine distance**) is often more effective than Euclidean distance. **5** Climate Change

present-day climate change includes both global warming—the ongoing increase in global average temperature—and its wider climate system

20 1 0

u v

effects on earth

The **cosine distance** between two vectors and is defined as: u · v

6 Renewable Energy

renewable energy also called green energy

is energy made

repl~~enished on a~~

human timescale

$\|u\|_2 \|v\|_2$

$\text{distance}_{\text{cosine}}(u, v) = 1 - (\cdot)$

Your tasks:

from renewable natural resources that are

20 1 0

7 Biodiversity biologists most often define biodiversity as the totality of genes

species and ecosystems of a region 16 1 0

Apply **DBSCAN** to cluster documents encoded with **sentence embeddings** (`emb_sents`) and store the cluster

8 French Cuisine french cuisine is the cooking traditions and practices of france 10 1 1 labels in `dbscan_emb_labels` .

9 Bread food bread is a baked food product made from water flour and often

Set `metric='cosine'` to use **cosine distance** for clustering.

yeast 13 1 1

Tune the **hyperparameters** `eps` and `min_samples` to obtain meaningful clusters, as the default values are unlikely

manti is a type of dumpling mainly found in turkish cuisine

10 Dumpling food but also in west asia

armenian cuisine and central asian cuisine 27 1 1

to work well on this dataset.

south caucasus and the balkans

Notes for tuning the hyperparameters:

Start by examining the minimum, maximum, and mean cosine distances in the dataset. You can compute pairwise cosine distances using the package imported below.

Experiment with a range of `eps` values and print the number of clusters and their sizes for each value.

Select the `eps` value that produces the desired number of clusters.

```
from sklearn.metrics.pairwise import cosine_distances
```

Solution_1.4

Points: 4

```
from sklearn.cluster import DBSCAN
```

```
dbscan = DBSCAN(eps=0.5, min_samples=1, metric = 'cosine')
dbscan.fit(emb_sents)

dbscan.fit_predict(emb_sents) # 11 clusters which is way too much
```

```
array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

```
cos_dists = cosine_distances(emb_sents) # pairwise cosine distances

print("min distance:", np.min(cos_dists))
print("max distance:", np.max(cos_dists))
print("mean distance:", np.mean(cos_dists))

# starts with eps=0.8
```

```
min distance: 0.0
max distance: 1.0320184
mean distance: 0.81524456
```



```
dbscan = DBSCAN(eps=0.8, min_samples=1, metric='cosine')
dbscan.fit(emb_sents)
labels = dbscan.fit_predict(emb_sents)

print(labels)

# choose eps=0.8 as it has 3 clusters
```

```
[0 0 0 0 1 1 1 1 2 2 2]
```

```
dbscan_emb_labels = labels


wiki_df["emb_dbscan"] = dbscan_emb_labels
wiki_df
```

Next steps: New interactive sheet

wiki	query	text	n_words	bow_kmeans	emb_kmeans	emb_dbscan
0	Artificial Intelligence	intelligence such as learning reasoning problem-solving perception and decision-making	25	1	2	0
1	Deep learning	in machine learning deep learning dl focuses on utilizing multilayered neural networks to perform tasks such as classification regression and representation learning	22	2	2	0
2	Unsupervised learning	in machine learning supervised learning sl is a type of machine learning paradigm where an algorithm learns to map input data to a specific output based on example input-output pairs	30	2	2	0
3	Quantum Computer	a quantum computer is a real or theoretical computer that exploits superposed and entangled states	15	1	2	0
4	Environmental protection	protection refers to the taking of measures to protecting the natural environment prevent pollution and maintain ecological balance	22	0	0	1
5	Climate Change	warming—the ongoing increase in global average temperature—and its wider effects on earth climate system	20	1	0	1
6	Renewable energy	renewable energy also called green energy is energy made from renewable natural resources	20	1	0	1

artificial intelligence ai is the capability of

keyboard_arrow_down

1.5 Hierarchical clustering with sentence embedding representation

rubric={accuracy}

Your tasks:

Apply hierarchical clustering on `emb_sents`. In particular

1. Create and show a dendrogram with `complete` linkage and `metric='cosine'` on this toy dataset.
2. Create flat clusters using `fcluster` with appropriate hyperparameters and store cluster labels to `hier_emb_labels` variable below.

Solution_1.5

that are replenished on a human biodiversity as the

timescale biologists most often define

7 Biodiversity totality of genes species and ecosystems of a region 16 1 0 1

Points: 3

8 French Cuisine french cuisine is the cooking traditions and

practices of france 10 1 1 2

9 B d f d bread is a baked food product made from water 13 1 1 2

9 Bread food b ead s a ba ed ood p oduct ade o ate

flour and often yeast 13 1 1 2

```
from scipy.cluster.hierarchy import linkage, dendrogram
from scipy.cluster.hierarchy import fcluster
```

10 Dumpling food manti is a type of dumpling mainly found in central asian cuisine but also in west asia south 27 1 1 2

turkish cuisine armenian cuisine and caucasus and the balkans

```
linkage_array = linkage(emb_sents, method="complete", metric="cosine")
ax = plt.gca()
dendrogram(linkage_array, ax=ax, color_threshold=0.8) # adjust threshold
plt.xlabel("Sample index")
plt.ylabel("Cluster distance")
```

```
Text(0, 0.5, 'Cluster distance')
```

```
hier_emb_labels = fcluster(linkage_array, 3, criterion="maxclust")
```

```
wiki_df["emb_hierarchical"] = hier_emb_labels
```



```
wiki_df
```

	wiki	query	text	n_words	bow_kmeans	emb_kmeans	emb_dbscan	emb_hierarchical
0	Artificial Intelligence		artificial intelligence ai is the capability of computational systems to perform tasks					
4	Environmental protection		environmental protection or environment protection refers to the taking of measures to protecting the natural					
1	Deep learning		in machine learning deep learning dl focuses on utilizing multilayered neural networks to perform tasks such as classification regression and representation learning					
2	Unsupervised learning		in machine learning supervised learning sl is a type of machine learning paradigm where an algorithm learns to map input data to a specific output based on example input-output pairs					
3	Quantum Computer		a quantum computer is a real or theoretical computer that exploits superposed and entangled states					
5	Climate Change		present-day climate change includes both global warming—the ongoing ~~increase in global average~~ temperature—and its wider effects on earth climate					

Next steps: New interactive sheet

system

renewable energy also called green energy is

6 Renewable Energy

renewable natural resources that are replenished on a human timescale

biologists most often define biodiversity as the totality of genes species and ecosystems of a region

16 1 0 1 3

7 Biodiversity

french cuisine is the cooking traditions and practices of france

10 1 1 2 1 13 1 1 2 1

8 French Cuisine **9** Bread food

bread is a baked food product made from water flour and often yeast

manti is a type of dumpling mainly found in turkish cuisine armenian cuisine and central asian cuisine but also in west asia south caucasus and the balkans

27 1 1 2 1

10 Dumpling food energy made from

20 1 0 1 3

keyboard_arrow_down 1.6 Discussion

rubric={reasoning}

Your tasks:

Reflect on and discuss the clustering results of the methods you explored in the previous exercises, focusing on the following points:

- effect of input representation on clustering results
- whether the clustering results match with your intuitions and the challenges associated with getting the desired clustering results with each method

Solution_1.6

Points: 4

Input representation effect

- In count vectorizer KMeans clustering did a very poor job at creating accurate clusters. This is because the countvectorizer uses a bag of words method where semantic meaning is not captured and similarity of clusters is formed based on similar frequency of words.
- Similar sentences were grouped together in sentence transformers, but very short or unusual sentences would appear in their own clusters
- Using cosine distance highlighted sentence meaning similarity better than Euclidean, which can be dominated by vector magnitude differences
- Choice of input representation directly affects cluster quality. If embeddings don't separate meaning well or distances in eps in euclidean are not sufficient, even the best clustering models will fail

Clustering results

KMeans

- Straightforward, fast and works well with this dataset given clusters of similar documents are evenly sized and distinct semantic groups
- After using sentence transformation, the KMeans performed very well and followed the clustering defined in exercise 1
- Needed to choose k (number of clusters)

DBSCAN

- Hyperparameter tuning of eps and min_samples is crucial to performance of DBSCAN
- Too low eps = too many single clusters, large eps = merging of unrelated points
- Min_samples = 1 was fixed, as there is not many data points anyway
- DBSCAN can give each wiki article its own cluster depending on eps, finding meaningful clusters required looking at max, min, mean pairwise cosine distances within dataset

Hierarchial clustering

- Provided full tree of merged clusters, can select number of clusters using dendrogram and does not require specifying number of clusters
- Choice of linkage will change cluster shape
- Dendrograms make it easy to visually interpret which sentences were similar
- Chose n_clusters = 3 because if it was too high, all points merge, but if it was too low, then every point was its own cluster

keyboard_arrow_down 1.7 Visualizing clusters

rubric={points:4}

One approach to working with unlabeled data is visualization. That said, our data is high-dimensional, making it challenging to visualize. Take sentence embedding representation as an example: each instance is depicted in 768 dimensions. To visualize such high-dimensional data, we can employ dimensionality reduction techniques to extract the most significant 2 or 3 components, and then visualize this low-dimensional data.

Given data as a `numpy` array and corresponding cluster assignments, the `plot_umap_clusters` function below transforms the data by applying dimensionality reduction technique called [UMAP] to it and plots the transformed data with different colours for different clusters.

> Note: At this point we are using this function only for visualization and you are not expected to understand the UMAP part.

You'll have to install the `umap-learn` package in the course conda environment either with `conda` or `pip`, as described in the [documentation].

```
> conda activate cpsc330
> conda install -c conda-forge umap-learn
```

or

```
> conda activate cpsc330
> pip install umap-learn
```

If you get an error with the import below try

```
pip install --upgrade numba umap-learn
```

Your tasks:

1. Visualize the clusters created by the methods above using `plot_umap_clusters` function below. In other words, visualize clusters identified by each of the methods below.

K-Means with bag-of-words representation

K-Means with sentence embedding representation

DBSCAN with sentence embedding representation

Flat cluster of hierarchical clustering with sentence embedding representation

```
import umap


def plot_umap_clusters(
    data,
    cluster_labels,
    raw_texts=None,
    show_labels=False,
    point_size=50,
    n_neighbors=15,
    title="UMAP Visualization",
    ignore_noise=False,
):
    """
    Perform dimensionality reduction using UMAP and visualize the resulting clusters.

    Parameters
    ----------
    data : np.ndarray
        Feature matrix to be reduced.
    cluster_labels : array-like
        Cluster label for each data point.
    raw_texts : list of str, optional
        Original texts corresponding to the data points (used when show_labels=True).
    show_labels : bool, default=False
        Whether to annotate points with text labels.
    point_size : int, default=50
        Size of the scatterplot points.
    n_neighbors : int, default=15
```

```
    Number of neighbors used in UMAP.
title : str, default="UMAP Visualization"
    Title of the plot.
ignore noise : bool, default=False
 g _ ,
    Whether to exclude points labeled as noise (-1).

Returns
-------
None
"""
reducer = umap.UMAP(n_neighbors=n_neighbors, random_state=42)
embedding = reducer.fit_transform(data)

df = pd.DataFrame(embedding, columns=["dim1", "dim2"])
df["cluster"] = cluster_labels

if ignore_noise:
    df = df[df["cluster"] != -1]

fig, ax = plt.subplots(figsize=(6, 5))
ax.set_title(title)

scatter = ax.scatter(
    df["dim1"], df["dim2"],
    c=df["cluster"], cmap="tab20b", s=point_size
)

ax.legend(*scatter.legend_elements(), title="Clusters", loc="best")

if show_labels and raw_texts is not None:
    for (x, y, text) in zip(df["dim1"], df["dim2"], raw_texts):
        ax.annotate(" ".join(text.split()[:10]), (x, y), fontsize=8)

plt.show()
```

Solution_1.7

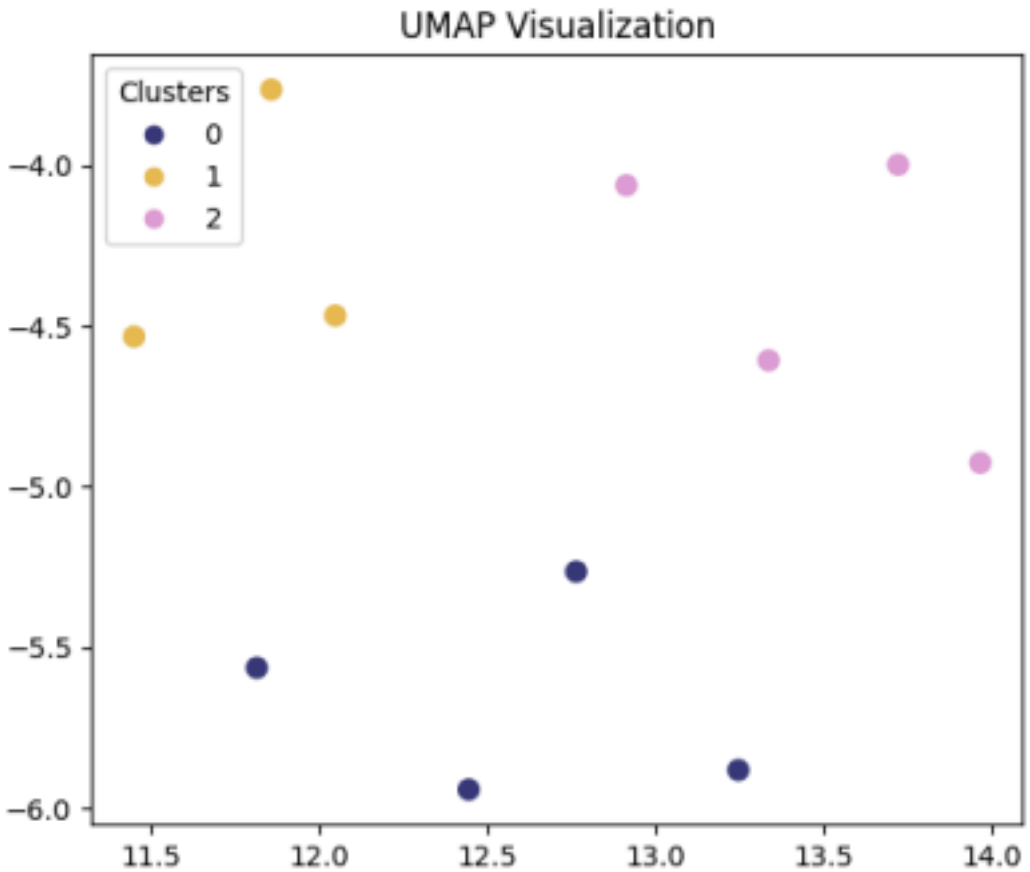
Points: 4

```
labels = [kmeans_emb_labels, dbscan_emb_labels, hier_emb_labels]
```



```
for label in labels:
    plot_umap_clusters(emb_sents, label)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by sett
warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:2462: UserWarning: n_neighbors is larger than the dataset
warn(
```




```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by sett
warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:2462: UserWarning: n_neighbors is larger than the dataset
warn(
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by sett
warn(
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:2462: UserWarning: n_neighbors is larger than the dataset
warn(
```

keyboard_arrow_down Exercise 2: [Food.com](#) recipes

Now that we have applied document clustering on a toy corpus, let's move to a more realistic corpus.

In the lecture, we worked on an activity of manually clustering food items and discussed challenges associated with it. We also applied different clustering algorithms to cluster food images. We'll continue this theme of clustering food items in this lab. But instead of images we will cluster textual description of food items, i.e., recipe names.

In this assignment, we will work with a sample of Kaggle's Food.com recipes corpus. This corpus contains 180K+ recipes and 700K+ recipe reviews. In this lab, we'll only focus on recipes and **not** on reviews. The recipes are present in `RAW_recipes.csv` . Our goal is to find categories or groupings of recipes from this corpus based on their names.

Your tasks:

Download `RAW_recipes.csv` and put it under the `data` directory in the homework folder.
Run the code below. The dataset is quite large, and in this assignment, for speed, you will work with a sample of the dataset. The function `get_recipes_sample` below carries out some preliminary preprocessing and returns a sample of the recipes with most frequent tags.

Note: Depending upon the capacity of your computer, feel free to increase or decrease the size of this sample by changing the value for *`n_tags`* . If you decide to go with a different value of *`n_tags`* , state it clearly in Exercise 2.1 so that the grader knows about it.

```
!ls
```

```
cache data hw6.ipynb img README.md
```

```
orig_recipes_df = pd.read_csv("data/RAW_recipes.csv", on_bad_lines='skip') # had to skip bad lines
orig_recipes_df.shape
```

```
(231637, 12)
```

```
def get_recipes_sample(orig_recipes_df, n_tags=300, min_len=5):
    orig_recipes_df = orig_recipes_df.dropna() # Remove rows with NaNs.
    orig_recipes_df = orig_recipes_df.drop_duplicates(
        "name"
    ) # Remove rows with duplicate names.
    # Remove rows where recipe names are too short (< 5 characters).
    orig_recipes_df = orig_recipes_df[orig_recipes_df["name"].apply(len) >= min_len]
    # Only consider the rows where tags are one of the most frequent n tags.
    first_n = orig_recipes_df["tags"].value_counts()[0:n_tags].index.tolist()
    recipes_df = orig_recipes_df[orig_recipes_df["tags"].isin(first_n)]
    return recipes_df
```

```
recipes_df = get_recipes_sample(orig_recipes_df)
recipes_df.shape
```

```
(9100, 12)
```

```
recipes_df["name"]
```

	name
42	i yam what i yam muffins
101	to your health muffins
129	250 00 chocolate chip cookies
138	lplermagronen
163	california roll salad
...	...
231430	zucchini wheat germ cookies
231514	zucchini blueberry bread
231547	zucchini salsa burgers
231596	zuppa toscana
231629	zydeco salad

9100 rows × 1 columns

dtype: object

In the rest of the homework, we will use `recipes_df` above, which is a subset of the original dataset.

keyboard_arrow_down 2.1 Longest and shorter recipe names

rubric={points:2}

Your tasks:

1. Print the shortest and longest recipe names (length in terms of number of characters) from `recipes_df` . If there is more than one recipe with the same shortest/longest length, store **one** of them in `shortest_recipe` and/or `longest_recipe` as a **string**.

Solution_2.1

Points: 2

```
lengths = recipes_df["name"].str.len()

shortest_recipe = recipes_df['name'][lengths == lengths.min()].iloc[0]
longest_recipe = recipes_df['name'][lengths == lengths.max()].iloc[0]
```

keyboard_arrow_down 2.2 More EDA

rubric={points:2}

Your tasks:

1. Create a word cloud for the recipe names. You can use the `wordcloud` package for this, which you will have to install in the course environment.

```
> conda activate cpsc330
> conda install -c conda-forge wordcloud
```


Solution_2.2

Points: 2

```
!pip install wordcloud
```

```
Requirement already satisfied: wordcloud in /usr/local/lib/python3.12/dist-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.12/dist-packages (from wordcloud)
(2.0.2 Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from wordcloud)
(11.3.0) Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from wordcloud)
(3.10.0) Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from
matplotlib->wo Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from
matplotlib->wordcl Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages
(from matplotlib->w Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->w Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->wor Requirement already satisfied: pyparsing>=2.3.1
in /usr/local/lib/python3.12/dist-packages (from matplotlib->wo Requirement already satisfied:
```

```
python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib Requirement already
satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->
```

```
from wordcloud import WordCloud
```

```
text = " ".join(str(name) for name in orig_recipes_df["name"])
wordcloud = WordCloud(width=800, height=400, background_color='white').generate(text)
```

keyboard_arrow_down 2.3 Representing recipe names

rubric={points:3}

The next step is creating a representation of recipe names.

Your tasks:

1. Similar to Exercise 1, create sentence embedding representation of recipe names (`name` column in `recipes_df`). For the rest of the homework, we'll stick to the sentence embedding representation of recipe names.

You might have to convert the recipe names to a list (`recipes_df["name"].tolist()`) for the embedder to work If you create a dataframe with sentence embedding representation, set the index to *`recipes_df.index`* so that the indices match with the indices of the sample we are working with.
This might take a while to run.

Solution_2.3

Points: 3

```
embeddings = embedder.encode(recipes_df["name"].tolist())
emb_recipe_df = pd.DataFrame(embeddings, index=recipes_df.index)
emb_recipe_df
```


	0	1	2	3	4	5	6	7	8	9	..
42	0.019592	-0.088336	0.072677	-0.034575	-0.048741	-0.049801	0.175334	-0.055191	0.020301	0.019828	.
101	-0.000567	-0.011825	0.073199	0.058176	0.031688	-0.015428	0.168134	0.000466	0.033078	-0.013923	.
129	-0.022604	0.065034	-0.033065	0.014450	-0.105039	-0.050559	0.100076	0.022929	-0.037398	0.011857	.
138	-0.066914	0.025988	-0.087689	-0.006847	-0.012861	0.049035	0.035351	0.124966	-0.011697	-0.050179	.
163	-0.007068	-0.007308	-0.026629	-0.004153	-0.052810	0.011126	0.024000	-0.036994	0.023526	-0.046870	.
...	...	...	...	...	...	...	...	...	...	...	.
231430	-0.036366	0.087173	-0.039641	0.002705	0.097142	-0.075385	0.068207	0.010435	-0.069214		

0.010464 . **231514** -0.052718 0.008980 -0.046014 0.030194 0.005201 0.009964 -0.006760 0.030238 -0.031474 0.024632 . **231547** -0.080801 0.004295 -0.044325 0.038308 -0.030125 -0.063566 0.004788 0.004822 0.015525 -0.040094 . **231596** -0.060801 0.111672 -0.108755 0.052323 -0.099851 -0.027532 0.084189 -0.004861 0.002891 0.013943 . **231629** -0.085059 0.009065 -0.088442 -0.008907 0.002234 -0.055801 0.039431 -0.037784 0.005315 -0.078361 . 9100 rows × 384 columns

keyboard_arrow_down Exercise 3: Clustering recipe names

In this exercise you'll cluster recipe names with some of the clustering algorithms we have seen in class. This will also involve making some attempts to pick reasonable hyperparameter values for each clustering method based on the quality of the resulting clusters. For example, for KMeans, you need to specify the number of clusters in advance, which is often

challenging on real-world datasets. For DBSCAN, you need to pick appropriate `eps` and `min_samples` . For hierarchical clustering, you need to pick a suitable linkage criterion, distance metric, and prune the tree so that it's possible to visualize and interpret it.

Here are some methods which may help you with picking reasonable values for the hyperparameters.

- Visualize the Elbow plot (KMeans).
- Visualize Silhouette plots.
- Visualize resulting clusters using `plot_umap_clusters` function from Exercise 1.
- Sample some recipes from each cluster, manually inspect whether there are coherent semantic themes. (For this, you may use the function `print_clusters` given below.)

You may use the `yellowbrick` package for visualizing the Elbow plot and the Silhouette plots. You can intall it with

```
conda install -c districtdatalabs yellowbrick
```

Note that the process of picking reasonable hyperparameter values will be exploratory, iterative, and will involve manual inspection and judgment, as there is no ground truth to verify how well the model is doing. In your solutions, please do not include everything you try. Only present the results of the most informative trials. Add a narrative to your answer so that it's easy for the grader to follow your choices and reasoning.

```
def print_clusters(recipes_df, cluster_labels, n_recipes=10, replace=False, random_state=None):
    """
    Given recipes_df containing recipe names and cluster assignment (labels),
    sample and print n_recipes recipes per cluster.

    Parameters
    -----------
    recipe_df : pandas dataframe
        recipes dataframe containing recipe names in the "name" column
    cluster_labels : ndarray or a list
        cluster labels for each row in recipes_df
    n_recipes : int
```



```
        number of examples to sample from each cluster
    replace: bool
        replace flag to pass to the sampling of recipe names

    Returns
    -----------
    None
    """

    grouped = (
        pd.DataFrame(
            {
                "name": recipes_df["name"],
```

```
                "cluster_label": cluster_labels,
            }
        )
        .sort_values("cluster_label")
        .groupby("cluster_label")
    )

    for name, group in grouped:
        print(f"Cluster {name}")
        print(("----------").format(""))
        print("\n".join(group.sample(n_recipes, random_state=random_state, replace=replace)['name'].tolist()))
        print("\n\n")
```

keyboard_arrow_down 3.1 K-Means

rubric={points:6}

Your tasks:

1. Cluster recipe titles using KMeans. Make some attempts to determine the optimal number of clusters.

2. Pick one or two best models and justify your choice.

Solution_3.1

Points: 6

Type your answer here, replacing this text.

```
kmeans = KMeans(n_clusters = 7, random_state = 123)
```

```
kmeans.fit(embeddings)

kmean_recipe_labels = kmeans.labels_
kmean_recipe_labels
```

```
array([1, 1, 5, ..., 2, 3, 2], dtype=int32)
```

```
print_clusters(recipes_df, kmean_recipe_labels)
```



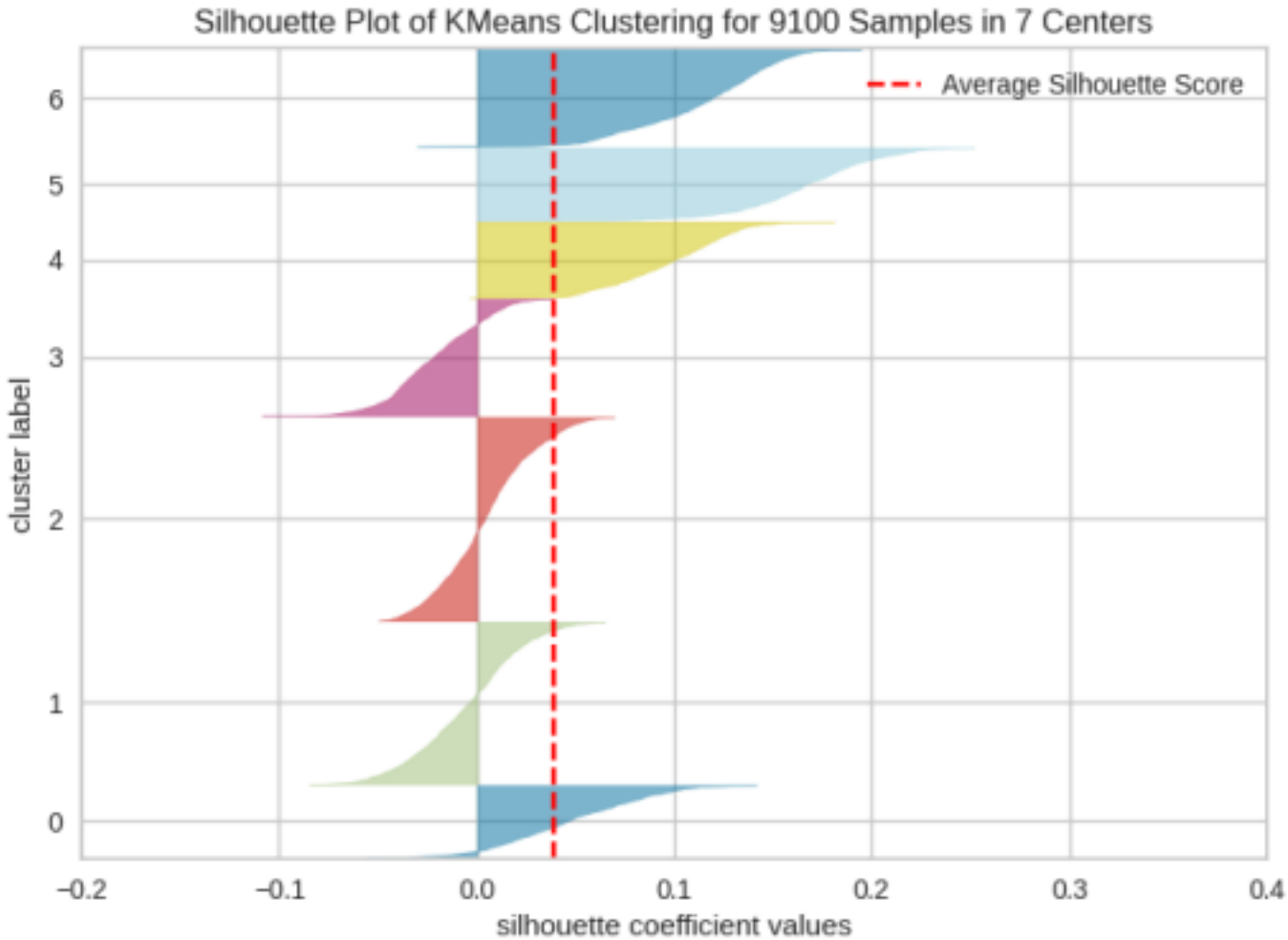
```
tequila lime chicken breasts
swiss chicken and ham casserole
chicken with prosciutto and bean puree
chinese brown sauce chicken
chicken ala king



Cluster 5
----------
peanut butter chocolate chip cookie cups
en light ened chocolate chip cookies
```

```
the best oatmeal cookies i ve found
weight watchers banana oatmeal cookies 1 pt
dad s cookies
yummy oatmeal raisin cookies
stefanie s gingerbread cookies
irish butter cookies
frosted molasses cookies
build your own healthy cookies



Cluster 6
----------
bailey s irish cream cheesecake
austin s favorite devil dog cake
almond cream cheese pound cake
orange babycake cheesecake
strawberry disaster cake
moist carrot cake
ma s oatmeal cake and icing
lemon buttermik cupcakes
cream cake
rainy day yellow cake
```

```
from yellowbrick.cluster import SilhouetteVisualizer
visualizer = SilhouetteVisualizer(kmeans, colors="yellowbrick")
```



```
visualizer.fit(embeddings)
visualizer.show()
```




```
<Axes: title={'center': 'Silhouette Plot of KMeans Clustering for 9100 Samples in 7 Centers'},
xlabel='silhouette coefficient values', ylabel='cluster label'>
```

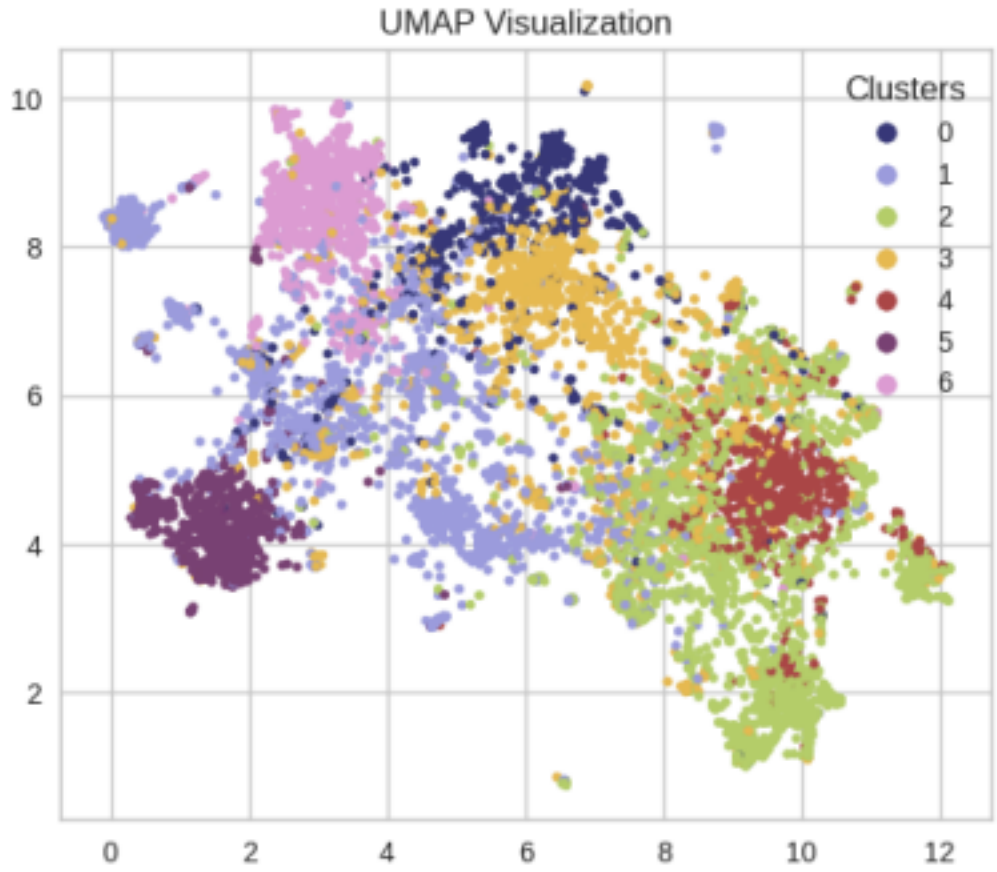
```
longest_recipe
```

```
'baked tomatoes with a parmesan cheese crust and balsamic drizzle'
```



```
plot_umap_clusters(embeddings, kmeans.labels_, point_size=10)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by sett
warn(
```



I chose n_clusters = 7 for KMeans. Initially, I began with 10 but when viewing the samples and the umap plot, it appeared to create clusters that were too specific and didn't generalize well. For example, chicken pasta was not in the broader'chicken' cluster and was more in the pasta cluster. The silhouette plot also was very sharp and had few samples in each cluster, indicating that clusters were becoming increasingly dissimilar. Reducing the number of clusters to 7 helped produce broader, more meaningful and coherent clusters that aligned better with my intuition about the data.

keyboard_arrow_down 3.2 DBSCAN

rubric={points:6}

Your tasks:

1. Cluster recipe names using `DBSCAN` with `metric="cosine"` . Make some attempts to tune the hyperparameters `eps` and `min_samples` .

Solution_3.2

Points: 6

Type your answer here, replacing this text.

```
dbscan = DBSCAN(eps=0.38, min_samples=5, metric = 'cosine')
dbscan.fit(embeddings)
db_labels = dbscan.fit_predict(embeddings)

len(set(db_labels))
```

```
7
```

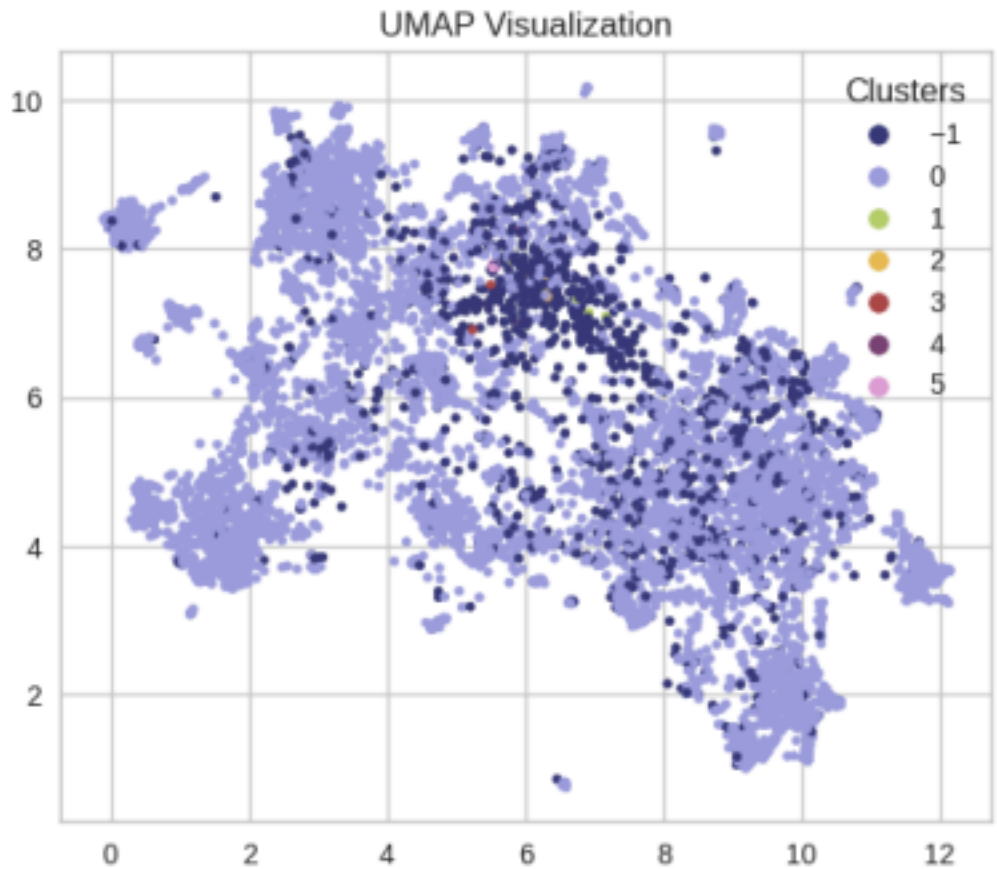
```
cos_dists = cosine_distances(embeddings) # pairwise cosine distances

print("min distance:", np.min(cos_dists))
print("max distance:", np.max(cos_dists))
print("mean distance:", np.mean(cos_dists))
```

```
# start with lower side eps = 0.3
```

```
min distance: 0.0
max distance: 1.2340795
mean distance: 0.7510912
```

```
plot_umap_clusters(embeddings, dbscan.fit_predict(embeddings), point_size=10)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by sett
warn(
```




```
print_clusters(recipes_df, db_labels, n_recipes=10, replace=3)
```

```
Cluster 5
----------
sex on the moon
sex on the moon
midori sex on the beach
sex on the moon
tgi friday s sex on the beach
midori sex on the beach
tgi friday s sex on the beach
frozen sex on the beach
creamy sex on the beach
sex on the moon
```

3.3 Hierarchical clustering

rubric={points:6}

Your tasks:

1. Try hierarchical clustering with `metric="cosine"` on this problem. Show a dendrogram by using a suitable truncation method.
2. Create flat clusters by cutting the tree at the appropriate level.

> *Note: Try orientation="left" of `dendrogram` for better readability of the dendrogram.*

Solution_3.3

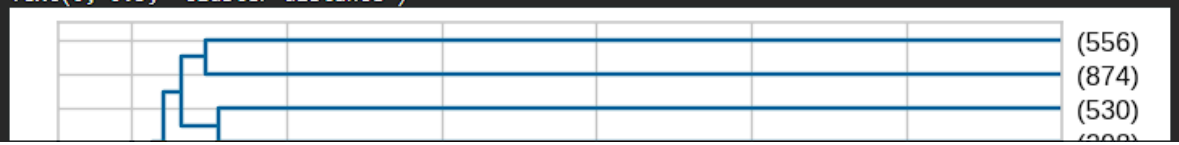
Points: 6

Type your answer here, replacing this text.

[50]
22s

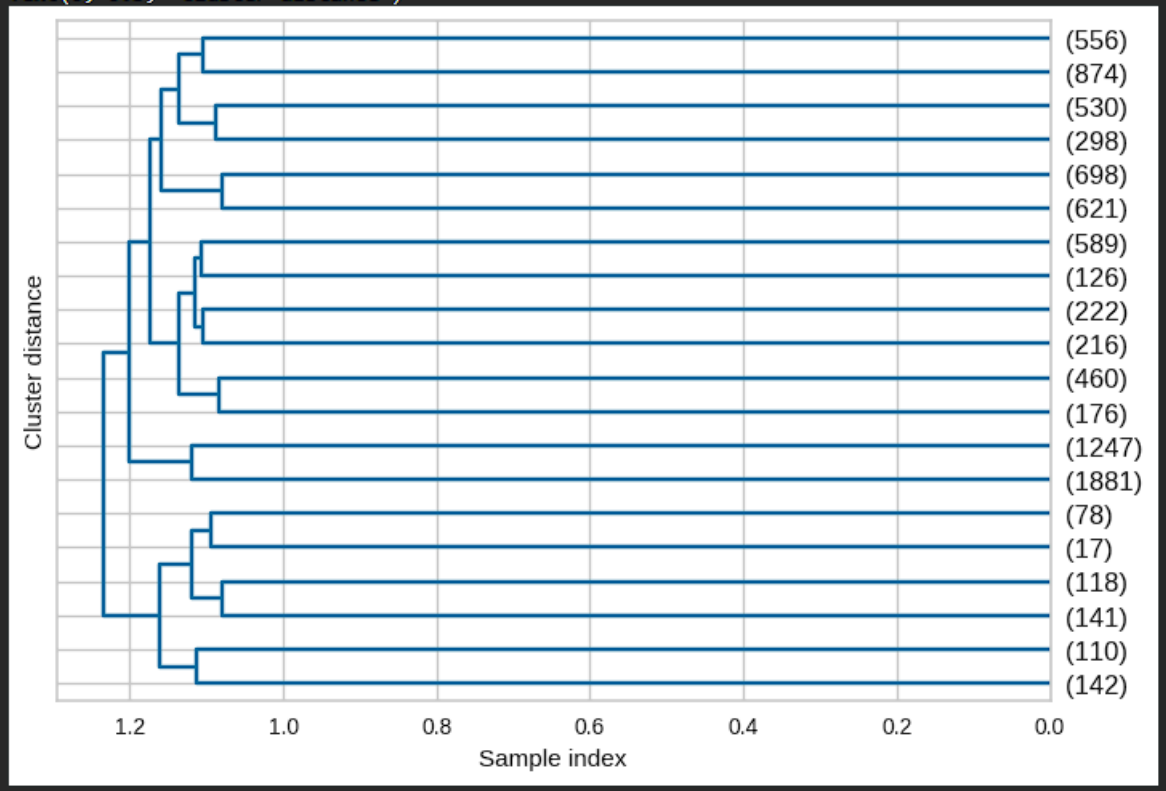
```
linkage_array = linkage(embeddings, method="complete", metric="cosine")
ax = plt.gca()
dendrogram(
    linkage_array,
    orientation="left",
    truncate_mode="lastp",
    p=20,
    color_threshold=0.8
)
plt.xlabel("Sample index")
plt.ylabel("Cluster distance")
```

```
Text(0, 0.5, 'Cluster distance')
```



```
plt.ylabel("Cluster distance")
```

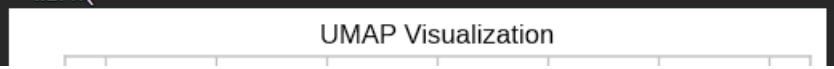
```
Text(0, 0.5, 'Cluster distance')
```

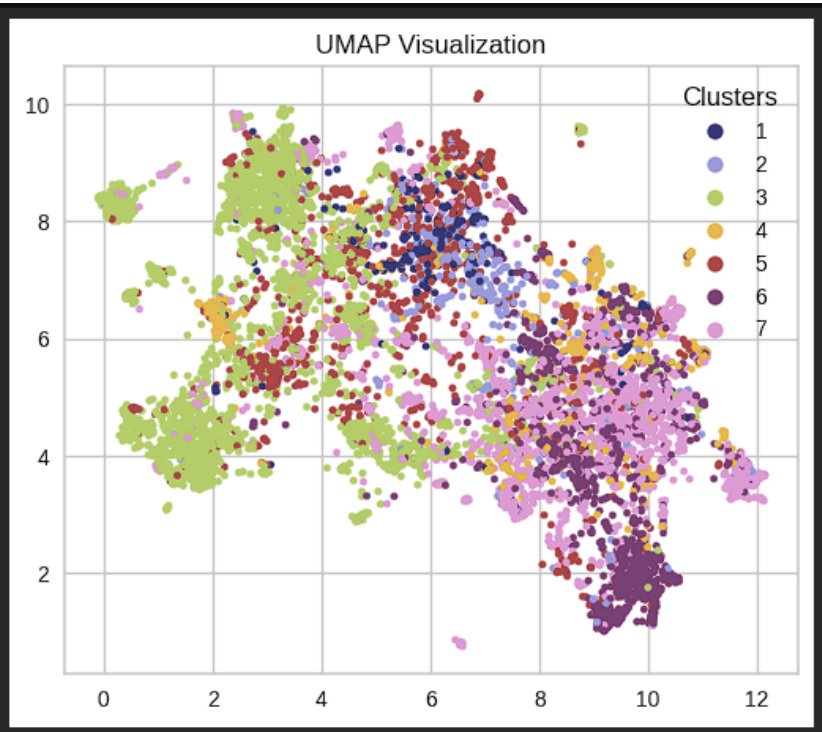



```
hier_recipe_labels = fcluster(linkage_array, 7, criterion="maxclust")
```

```
plot_umap_clusters(embeddings, hier_recipe_labels, point_size=10)
```

```
/usr/local/lib/python3.12/dist-packages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed for parallelism
  warn(
```





[53]
0s

```
print_clusters(recipes_df, hier_recipe_labels)
```

```
Cluster 1
----------
mussakhan  baked chicken on bread  middle east  palestine
passion potion
freddy fudpucker
lemon berry cooler
matcha green sparkler
sea breeze salad
afterburner
carnitas  simple but amazing
gallstone shooter
b 52  layered shot
```

3.4 Manual interpretation of clusters

rubric={points:6}

Your tasks:

1. Label the topics/themes you see in the clusters created by different clustering methods.
2. Do you see a common theme across clusters created by different clustering methods? Do you see any differences between the clusters created by different clustering methods?

Solution_3.4

Points: 6

After examining the clusters by the different methods, several themes appeared across clusters. Many clusters grouped recipes based on main dishes or dish types, chicken, cookies, cakes, as long as it was in the name of the recipe, it would be grouped together. These themes appeared relatively consistently across clustering methods. Suggesting that sentence embeddings can capture semantic similarities across recipe names and their ingredients. KMeans clusters tended to be way more balanced in size because every recipe had to be assigned to a predetermined cluster. The produced clusters were broad and were grouped into general categories like 'cakes,' 'chicken,' 'drinks.' DBSCAN had a difficult time getting clusters, I tried to tune hyperparameters to get 7 clusters, but when plotting the UMAP plot it revealed one large cluster. This may be because DBSCAN does not force clusters like KMeans does, and relies on density of data points in a space. Since many recipe embeddings can be close to each other in the embedding space, it makes it difficult to separate them into divided clusters. For hierarchial clustering, the dendrogram showed branches merging at high distances, this suggests that many recipes are dissimilar to each other and merge into larger cluster at higher distance thresholds. I vaguely spotted 5-7ish clusters I could do, and since KMeans clustering had the best at 7, I decided on the latter. While each method differed in how they clustered data, they generally revealed similar themes in recipe types and ingredients.